AI Assisted Furniture Planner: An A* Search Algorithm

CPSC 481

Professor Hamid Ebrahimi
Written by Robin Dunne

1. Problem Statement

Shopping for furniture can be burdensome due to the amount of time and energy it takes. Having to manually compare prices, styles, sizing, and much more can be slow and doesn't come naturally to everyone, which is why it often leads to inconsistent themes and coloring throughout the home.

This project aims to address these issues by providing an all in one planner that turns a description of a home into a curated furniture bundle. Given a description such as "sleek apartment for a software engineer who likes darker tones", the algorithm can detect key words and compare relevancy to pieces of furniture. There are several characteristics to what makes a piece of furniture suitable for a customer, including its relevance to the user's:

1. Style
2. Budget
3. Resident Attributes

The ability of a human to perceive different combinations of furniture for their homes and find the optimal outcome is not realistic, and therefore the core of this issue is most suitable to an AI resolution.

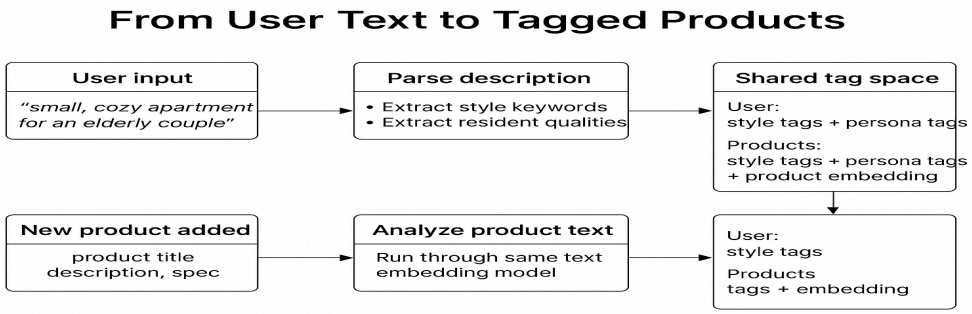
2. The Approach

There are three main parts to the approach:

1. Converting user text into internally mapped style and "persona" tags.
2. Defining a heuristic function that measures how well each piece of furniture, including its allocated tags, coincides with the tags extracted from the user text.
3. Implementing two planners, including a baseline Greedy Planner, and an A* Search Planner. We define two so that we can compare how much better or worse our interpretation is.

2.1 Text to Tags

The system begins by prompting the user to type a short description of what they want the house to be like and who will be living there. This sentence is parsed to extract style and persona tags. In the full implementation, both the user text and the product descriptions are passed through the same text-embedding model, improving the semantic matching between the user text and a product's attributes.



2.2 Heuristic Function

Each potential product candidate is assigned a quality score between 0 and 1, and is made of three different components:

1. Style Score - 0.5 / 1.0 - the fraction of the user's style tags that appear in the product's style tags times its importance, 0.5.
2. Persona Score - 0.35/1.0 - the fraction of the persona tags that appear in the product's persona tags times its importance, 0.35.
3. Price/Area Score - 0.15/1.0 - the measure of an item's "budget-friendliness" and space efficiency. To calculate this weight, we first determine the average size and price of the category of furniture that we are looking at, and then determine how close the price and size of our item is to the average. If it is 25% less than the average, then the heuristic is 1.0. Likewise, if it is 25% over, the heuristic is 0.0.

$$H = w_{\text{style}} \cdot \text{style_score} + w_{\text{persona}} \cdot \text{persona_score} + w_{\text{price}} \cdot \text{price_area_score}$$

Where Wstyle = 0.5, Wpersona = 0.35, and Wprice = 0.15. These weights were chosen to favor style and personal attributes the most, although price and size are still important constraints. Price and size will be hard constraints when determining the optimal bundle.

2.3 Greedy Baseline Planner

The greedy planner fills a room slot-by-slot. It processes required slots first (ensuring the minimum viable room), then attempts optional slots if budget and area remain. For each slot, it fetches candidates, applies hard constraint filters (remaining budget and remaining area), and selects the item with the best effective-price key (lower is better). Because it commits to each choice immediately, it cannot backtrack or trade off across slots; an early choice can block better global combinations.

2.4 A* Planner

In an effort to overcome these limitations, this project works to implement an A* Planner that searches over partial room configurations over independently filled slots. The state of the algorithm represents:

1. How many slots have been filled.
2. The list of products chosen so far.
3. Total cost and total area used.
4. Total quality score of the chosen items.

This planner uses a priority queue of states ordered by a best estimate of the potential total quality each could achieve. For a state with current score, g, and position, i, the planner computes an upper bound on the remaining achievable score. Then, where h(i) = sum of the best achievable match scores for the remaining slot instances, the algorithm prioritizes states by g +

h(i). This algorithm always prioritizes states that have the highest potential total score if all remaining slots were filled optimally.

At each step, the A* planner:

1. Pops the state with maximum g + h(i) from the queue.
2. If it violates the budget or area, it is pruned.
3. If all slots are filled, the state is complete, and therefore returns it as the best solution.
4. If not, it expands the state by trying to reach a candidate for the next slot that still fits the remaining budget and area, creating new states and pushing them into the queue.

In a perfect world, there would be no worry of optimization, but unfortunately we need to keep runtime manageable. We do so by:

1. Capping the number of candidates per slot, 50.
2. Using the upper bound h to prune any state that cannot possibly defeat the current best solution.

The A* Planner reliably finds higher quality furniture bundles than the greedy, while still running incredibly efficiently.

3. Software Description

The system is implemented as a command-line demo that collects user input, extracts tags, fetches slot-specific candidates from a product catalog, and then runs both planners to produce room bundles.

3.1 System Components

1. Product catalog (Supabase/PostgreSQL): furniture rows with price, estimated area, and tags.
2. planner_greedy.py: data models (BundleItem, BundleResult) and greedy planner.
3. planner_astar.py: SearchState, scoring functions, and A* search loop.
4. planner_demo.py: CLI driver that runs both planners and prints results.

3.2 Data Flow

Figure 1 summarizes the end-to-end pipeline from user input to final room bundle output, and shows where tag extraction, candidate retrieval, and planning occur.

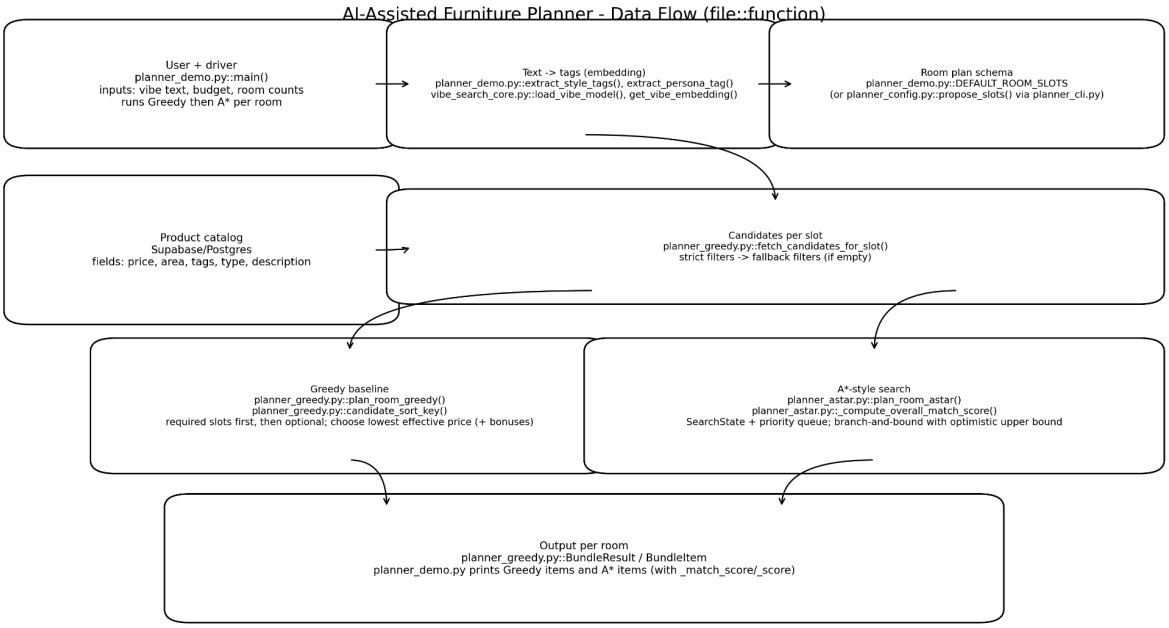


Figure 1: End-to-end project flow with concrete entrypoints (file::function calls used in the CLI demo).

4. Evaluation

Setup: 10 runs per planner on the same home request: "Sleek apartment for a software engineer who hates clutter." Total furniture budget: $1500. Rooms: 2 bedrooms, 1 living room, 1 bathroom, 1 kitchen, 1 office.

Metrics: Runtime per room (seconds) and bundle quality. Quality is computed as the average item match score H across items selected for a room, then averaged across rooms and runs. A score of 1.0 indicates strong alignment with the user's style/persona and favorable budget/area usage; 0.0 indicates poor alignment.

Planner	Avg runtime per room (s	Best-case runtime (s)	Avg quality score
Greedy	0.93	0.47	0.38
A*	1.03	0.49	0.72

Table 1: Planner performance summary (10-run average). A* achieves higher bundle quality with a small runtime increase compared to Greedy.

As shown in Table 1, A* improves average quality from 0.38 to 0.72 (nearly 2x) while increasing runtime by about 0.1 seconds per room on average.

5. Conclusion and Future Work

This project demonstrates that search-based planning can generate coherent, constraint-aware furniture bundles from a short natural-language description. The greedy baseline is fast but limited by local decisions, while the A* planner produces significantly higher-quality room bundles with a modest runtime cost.
Future work: The most compelling real-world use case is likely secondary markets (Facebook Marketplace or Craigslist), where users care more about strict budgets and finding affordable alternatives. A next step would be evaluating the feasibility of ingesting or scraping Marketplace listings and using the planner to recommend cheap, style-matched items. Additionally, a simple Swift frontend could provide a text box prompt ("How can I help you?") plus steppers for room counts and furniture-per-room selection.

6. References

• Russell, S. and Norvig, P. Artificial Intelligence: A Modern Approach (A* search and heuristic search).
• Python Standard Library documentation: dataclasses and heapq.
• Supabase documentation (database and query layer).
• IKEA product catalog pages used to populate the furniture database.